Emending Nonlinear Recurrence

Erik Garrison
erik.garrison@gmail.com
Poietic PBC / UTHSC, Memphis, TN, USA

Abstract

Emender makes recurrent memory a place for ongoing computation: each input corrects the memory's current prediction and applies a nonlinear transformation to the state carried forward. Width-axis multi-programming runs many small memory heads in parallel while preserving each head's sequential computation. At 1.3 billion parameters, E97 trains through 150.8 billion Pile tokens on one eight-GPU workstation and reaches comparable training loss to a tightly parameter-matched GDN2-MLP control. Fine-tuning these same models on an iterated nonlinear map gives E97 99.96% versus 76.09% accuracy at the trained length and 76.07% versus 53.36% at eight times that length. This supports the hypothesis that repeated nonlinear state updates provide a useful computational bias. On the distinct S_5 group-state task, GDN2-MLP leads the long-horizon comparison; both models' accuracy declines with length. Lean 4 results characterize nonlinear temporal composition and the conditioned affine GDN state channel, with additional update-family and routing results under explicit hypotheses. They do not establish a full-stack impossibility. The study is single-seed; the iterated-map comparison uses a common fine-tuning cap and stopping rule with unequal realized steps. Held-out language-model evaluation is pending. The contribution is a practical nonlinear-memory architecture whose learned computational behavior can be tested against its mathematical mechanism.

Keywords recurrent neural networks · language modeling · nonlinear recurrence · state-space models · expressivity · Lean 4

1 Introduction

A sequence can require more than a record of what arrived. Its elements may be instructions that change a running calculation, measurements that alter a feedback response, or observations that revise an earlier association. In each case the current state is part of the next computation. A recurrent model provides a natural setting for this process: it reads an input, updates its internal state, and carries that state into the following step. The question is what computation the update performs, and whether an update rich enough for these tasks can also be trained efficiently at language-model scale.

Emender addresses this question with a nonlinear matrix memory. At each step, a memory head reads its current prediction at an input-supplied key, writes a correction toward the supplied value, and passes the updated state through tanh. The correction makes the write depend on what the memory already contains. The nonlinearity then transforms the state that the

next step will receive. A long sequence therefore executes a long composition of nonlinear state updates, even when the network has a fixed number of layers. This describes the executed computation; it does not establish that no shallower representation of its result exists.

An *emender* is one such recurrent layer, and the *Emender* is the family of networks formed by stacking these layers. The current instance, *E97*, adds separate key-axis erase/read and value-axis write gates to the delta-correcting cell. Each head has its own small matrix state; projections, gates, and a channel-mixing MLP connect the memories to the rest of the model (Section 3). The intended memory is revisable: a later input can correct an earlier association. Saturation also motivates a form of persistence, but the available latching theorems are conditional slot-level results, including a hold hypothesis outside the production decay range, rather than a persistence guarantee for the trained model (Section 7).

The computational choice creates a systems problem. Modern linear-state recurrences, including state-space models and gated delta networks, gain training throughput by composing their state updates in parallel across time [1], [2], [3], [4]. A nonlinear update generally loses that scan structure. We instead use *width-axis multi-programming*: each head processes time sequentially, while many heads and batch elements run side by side. Small state tiles make these recurrent programs cheap enough to keep on the accelerator. This route preserves nonlinear state computation without adding attention or parallelizing the time loop (Section 4). It complements approaches that interleave recurrent layers with attention or parallelize nonlinear recurrence through iterative solvers [5], [6].

The first empirical question is whether this route is practical. E97 trains through 150.8 billion tokens on The Pile [7] on a single eight-GPU workstation. Its 1.3 B-parameter GDN2-MLP control is within 0.01% of its parameter count; both use the same tokenization, 2,048-token chunks, optimizer, and local DiLoCo recipe, with geometry and learning rate selected separately by CMA-ES. At matched tokens, the two occupy the same training-loss band, and the sign of their small loss difference depends on the averaging window (Section 5). This is a viability result at one scale, not a training-speed victory or a held-out equivalence result. The current pair's held-out evaluation remains pending. The empirical argument here concerns this matched pair.

The second question is what these same trained models can learn to do with their states. Bulk language-modeling loss leaves that question unresolved. An iterated nonlinear map asks the model to repeatedly transform a running value, directly testing the computational pattern that motivates E97. After fine-tuning for a modular-quadratic recurrence, E97 reaches 99.96% versus 76.09% accuracy at the trained length and 76.07% versus 53.36% at eight times that length. Its higher accuracy and greater length robustness support the hypothesis that a nonlinear operation in the carried-state path is a useful inductive bias for repeated nonlinear computation. The comparison is single-seed with a common cap and stopping rule, unequal realized training, and declining accuracy with length; it is evidence for that mechanism hypothesis, not proof of unique algorithm recovery.

The S_5 group-state task asks a different computational question: can a model learn to compose a finite set of permutation actions? GDN2-MLP leads the current pair's long-horizon comparison. This strong control result belongs in the expressivity analysis: it measures learned tracking and learning efficiency on a particular task, rather than implying a universal hierarchy between the full architectures. The iterated-map and group-state protocols, results, and qualifications are reported together in Section 6.

The formal analysis separates three questions that the experiments alone cannot settle: what a state update computes, what an update family can represent, and what a trained model actually learns. Lean 4 verifies that E97 executes one more nonlinear state map per token, whereas a GDN fold with a fixed, state-independent parameter schedule remains affine in its initial state. This is a statement about the conditioned state channel, not global linearity of a multilayer neural network. Additional results identify the adjacent split-gated cores and describe the S_5 tracker and routing conditions. The proofs do not supply a full recurrent tanh tracker, a minimum-depth bound for the trained stack, or a blanket linear-state impossibility (Section 7).

The resulting contribution is an architectural option with a concrete experimental program centered on a matched E97/GDN2-MLP pair. Nonlinear memory computation can be trained at 1.3 B scale with comparable language-modeling loss; the iterated-map advantage supports its computational motivation; and formal statements identify the state-channel structure relevant to that hypothesis. Feedback systems, iterative optimization, and driven dynamics give reasons to study this option beyond language modeling, while their application-level benefits remain to be tested (Section 8). We proceed from the state-update distinction and architecture to systems, language modeling, and computational probes, then state the formal boundaries and broader applications. Section 10 collects the unresolved questions for the current comparison.

2 Background

The central distinction is where nonlinear computation occurs. A model may apply nonlinear projections and gates to each input while keeping its carried-state update affine. Another may apply a nonlinearity to the state itself at every step. The two designs support different execution strategies. This section defines that distinction and the state-tracking tasks used to examine its consequences; circuit claims are kept within their stated arithmetic and architectural assumptions.

Long biological sequences are one motivation for the program. Pangenomic datasets run to terabases per study [8], [9], [10], creating pressure for sequence models with manageable state and training cost. This motivates the search for efficient recurrent computation, but does not establish that any particular pangenomic task requires nonlinear-state recurrence. Language modeling and controlled computational probes provide the measurements in this paper; biological applications remain future work.

Notation

The same state-channel symbols are used in the architecture, applications, and formal sections. Head and time subscripts may be omitted for a single step. A prime denotes the returned state, not a derivative. Matrix products and scalar scaling use juxtaposition; $\odot$ denotes componentwise multiplication, and T in a superscript denotes transpose.

Symbol	*Meaning and source-name correspondence*
S, S_h, S_t, S'	Carried state, head state, time-indexed state, and returned state of E97 and GDN. Lean names this matrix `H`.
H, H_t	M²RNN comparator state only; H also denotes the head count when explicitly described as a count.

Symbol	*Meaning and source-name correspondence*
N, V, d	Key-axis and value-axis dimensions; square formal state dimension $d = N = V$. Lean key dimension is `K`.
k, q, v	Write key, read query, post-activation value injection.
e, w, g	Key-axis erase/read gate (Lean `b`), value-axis write gate, output gate. b_t in an affine fold is an additive offset, not the erase gate.
δ, r	Value-vector correction and retrieved value in the cell.
d_h, λ, D	Production per-head decay, abstract scalar decay, and GDN pre-decay matrix. d_h plays λ's role; the formal E97 fold fixes λ, while production decay depends on the input. In the contractive control, λ instead denotes a Lyapunov exponent.
x_t, u_t, θ	Token/layer input, generic input injection, and fixed update-law parameters; v is the cell's value injection, not every application's input.
t, T, h, ℓ, (i, j)	Step, sequence length, head, layer, and matrix-slot indices. Local latch threshold T is not length.
ω, η, ε, M, μ	Slot write contribution, saturation slack, post-update slack, perturbation bound, and release margin. Scalar perturbation δ in set F is not the value vector.
s, r_t, w_t, g_t, z_j, h_t, n_t, x_t	Local states: group element, converter residue, filter coefficients, log-gain, optimization iterate, reservoir, population, and plant. These application-specific coordinates instantiate generic state S; x_t also denotes the iterated-map task state in the probe sections.

Linear-state and nonlinear-state recurrence

The recurrent language model lineage runs from Elman networks [11] and the LSTM [12] through the modern linear-recurrent variants: state-space models such as Mamba and Mamba2 [1], [2], gated linear attention [13], and the delta rule [14]. Many modern designs favor time-axis parallelism via linear-state updates, whereas nonlinear-in-time recurrence generally resists that scan. This structural tradeoff is not a universal historical claim of declining expressive power across the lineage. Gated DeltaNet-2 [4], the split erase/write-gate extension of Gated DeltaNet [3], supplies the linear-state mixer in the current GDN2-MLP control (Section 5). Mamba-3 [15] is reported by its authors to outperform GDN; its compilation-heavy HPO does not fit the Section 5 matched protocol, so it is left to future work.

A single explicit criterion classifies recurrent architectures. A recurrent layer is *linear-state* if its update can be written $S_t = A_t S_{t-1} + b_t$ with A_t and b_t depending on the current input x_t only. Here the state is expressed in vector coordinates; a matrix state may be vectorized for this criterion. Otherwise, if the previous state S_{t-1} appears nonlinearly in its own update equation, the layer is *nonlinear-state*. The distinction is structural; it does not depend on whether the non-linearity is a tanh, an exponential, a tanh-of-product, or a sigmoid.

Linear-state designs admit a chunkwise or scan-based parallel implementation along the time axis because composing $S_T = A_T S_{T-1} + b_T = \ldots$ unfolds into a product of inputs only. This is the reason linear-state recurrence dominates the billion-parameter landscape. Mamba,

Mamba2 (SSD), RetNet, GLA, DeltaNet, Gated DeltaNet, RWKV-4/5/6/7, HGRN2, mLSTM, MinGRU/MinLSTM [16] and Griffin's RG-LRU [17] are all linear-state by this criterion. At fixed precision and width, both state regimes are finite-state recognizers. Circuit-complexity containment results for restricted linear SSMs require additional arithmetic and transition assumptions [18]. Containment in TC^0 alone does not prove inability to solve an NC^1-complete word problem: that conclusion would require the unproved separation $TC^0 \neq NC^1$. In particular, affinity alone does not prohibit a finite-group action; realizability in the constrained GDN channel remains open (G3, Section 10). For fixed-depth transformers in the same formal-language setting, this is also the relevant ceiling under explicit arithmetic assumptions: log-precision transformers are captured by first-order logic with majority quantifiers [19], and average-hard attention transformers exactly, plus softmax-attention transformers with $O(\text{poly}(n))$ floating-point precision or $2^{-O(\text{poly}(n))}$ absolute-error approximation, lie in DLOGTIME-uniform TC^0 [20]. The classical empirical witness is the symmetric-group S_5, which has 120 elements and is the smallest non-solvable symmetric group.

Nonlinear-state recurrences include the classical LSTM and GRU; the sLSTM block of xLSTM [21] (memory-mixing through the hidden state inside exponential gates); M²RNN [5] (matrix state inside tanh with raw-write injection); Titans [22] (MLP memory with online gradient updates); and the architecture introduced here. The Siegelmann–Sontag result [23] concerns nonlinear recurrent nets under its own precision and resource assumptions, not Turing-completeness of a fixed-width, fixed-precision E97. Barrington's theorem identifies the non-solvable S_5 word problem as an NC^1-complete witness; it does not separate NC^1 from TC^0.

Matrix state

Replacing a vector hidden state $h \in \mathbb{R}^d$ with a matrix state $S \in \mathbb{R}^{N \times V}$ is not specific to the Emender. Linear-state designs (mLSTM, RWKV-5/6, DeltaNet) already use matrix or expanded states; RetNet's accumulation $S_t = \gamma S_{t-1} + k_t v_t^T$ is matrix-valued. The point of matrix state is that an outer-product update $S_t = ... + k_t \delta_t^T$ provides $O(NV)$ scalars of dynamic state at $O(NV)$ computational cost per token, with content-addressable retrieval via $S^T q$. Matrix state is common to the post-Mamba landscape (Mamba2 with `d_state`, GLA, DeltaNet, RWKV-5/6/7, mLSTM, M²RNN) and is treated here as a precondition shared by the Emender and its baselines, not as a contribution.

The S_5 state-tracking probe

The symmetric group S_5 is the group of permutations of five items; it has 120 elements and is the smallest non-solvable symmetric group. Its *word problem* is the following task: the model reads a sequence of permutations — here, adjacent transpositions, the swaps of two neighboring items — one at a time, and after each one must report the running composition of everything read so far, its *prefix product*. By Barrington's theorem [24] this task is complete for the complexity class NC^1. Exact recognition over all words, not finite-length accuracy, is the complexity-theoretic question; a failure to learn at training length does not locate a model in a smaller circuit class. The solvable-group control S_3 (permutations of three items, 6 elements) is included to factor out the part of difficulty that comes from prefix tracking *per se* rather than from non-solvability.

A dynamics taxonomy of head-types: eigenvalue placement × saturation

The linear/nonlinear hinge above, and the choice of where a head's state may decay, reflect, or oscillate, are two independent axes of a single design grid. The *dynamics* of a head's per-token state map locate it in a two-axis grid:

1. *Eigenvalue placement in the unit disk.* Where the along-key eigenvalue of the per-token state transition sits in the complex unit disk. A real-positive eigenvalue is a *decay* — the stored value fades toward zero along the key. A real-negative eigenvalue is a *reflection* that flips sign each step (`reflect`); reaching it is the lever that lets a recurrence track non-solvable group structure such as S_5, within the restricted transition analyses motivating this grid, not from affinity alone. A complex-conjugate pair is a *rotation* (`rot`) that oscillates the stored value. Real-positive and real-negative are the two ends of the *real diameter*; complex placement walks off that diameter into the rest of the disk.
2. *State map: linear or saturated.* Whether the state update is left linear or passed through a bounded saturating nonlinearity (canonically hardtanh; the `-nonlin` axis; the Emender uses the smooth variant tanh). Saturation latches a driven slot and makes the head finite-state, the boundedness-not-counting regime of Section 2.

The grid these two axes span names every Emender head-type (Figure 1). Real × linear at the positive end is `decay`, which is vanilla Gated DeltaNet; at the negative end it is `reflect`, Gated DeltaNet permitted a negative eigenvalue. Real × saturated is `nonlin` — the saturated delta-correcting head, historically tagged E97, whose split-gated instance is the 1.3 B model of this paper with split erase/read and write gates. Complex × linear is `rot`, the complex-eigenvalue oscillator head. Complex × saturated is `rot-nonlin`, a nonlinear oscillator left to future work, included as the combination of the two axes. E97 identifies the trained instance; the architecture vocabulary is this grid.

The adjacent split-gated cores locate the linear-recurrent baseline in this grid.

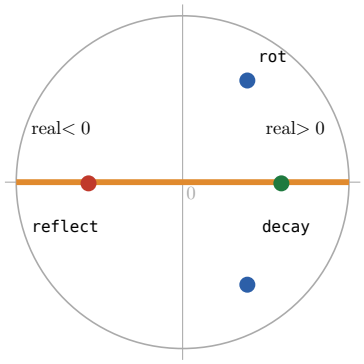
Shared split-gated cores, with different decay placement. Gated DeltaNet-2 and E97 have adjacent split erase/write cores, but deleting tanh at otherwise equal parameters does not identify them: E97 uses $\lambda S - k(e \odot k)^T S + k(w \odot v)^T$, whereas GDN-2 uses $(I - k(e \odot k)^T)DS + k(w \odot v)^T$. The exact Lean identity applies E97′s unit-decay linear core to the *pre-decayed* state DS; with identity decay the cores coincide (Section 7). No whole-production-stack inclusion or production-domain parameter remapping is proved. The eigenvalue/saturation grid is a descriptive head taxonomy, not such an inclusion theorem.

3 Computing with a nonlinear memory

Per-head update

Section 1 named the emender layer by what it does — read what the memory predicts at the addressed slot, compute the prediction error, and write the correction — and named E97 as the 1.3 B instance carried through the paper. This section makes the layer concrete: the per-step computation, the choices that fix it, and the shape numbers that make E97 one specific member of the family.

A. Eigenvalue placement in the unit disk



Real diameter (orange) = GDN-2; off-axis = rotation.

B. Head-type grid: eigenvalue placement × saturation

	decay (real > 0)	*reflect* (real < 0)	*rot* (complex)
linear	`decay`	`reflect`	`rot`
+ hardtanh (`-nonlin`)	`nonlin`	`nonlin`	`rot-nonlin` (future)

Shading: GDN-2-adjacent linear cores; decay placement differs.

Figure 1: *The Emender head-type taxonomy: eigenvalue placement × saturation. (A)* Each head is fixed by where its along-key eigenvalue sits in the complex unit disk — real-positive `decay`, real-negative `reflect`, or complex `rot` — and by whether the state map is linear or saturated. The orange real diameter is the locus GDN-2 occupies. *(B)* The two axes name five cells. Shading marks GDN-2-adjacent linear cores; the Emender also explores off-axis rotation (`rot`) and saturated state maps (`nonlin`) as a within-layer pool. This taxonomy is descriptive: different decay placement prevents an exact equal-parameter restriction of the production cores. `rot-nonlin` is reserved for future work. The GDN-2 corner of this grid is not a strawman: it is the mixer of the 1.3 B linear baseline of Section 5 (GDN2-MLP), selected by the same search protocol as E97.

An emender layer runs H independent heads side by side. Each head h owns its own matrix-state memory $S_h \in \mathbb{R}^{N\times V}$ — the matrix state of Section 2, here addressed by an N-dimensional key and holding a V-dimensional value at each addressed slot. (Section 7′s resource theorems use a single symbol d for the common matrix dimension; in E97, $d = N = V = 32$, so each head's memory is a 32×32 tile.) At each token, the layer's input projections supply, per head, a key k_h and query q_h in $\mathbb{R}^N$ that name the slots to write and read, a value $v_h \in \mathbb{R}^V$ to store, a scalar decay $d_h \in (0, 1)$ that fades the old state, and an output gate $g_h \in \mathbb{R}^V$. E97 controls its write with two input-dependent gates, split along the two axes of the matrix state: a key-axis *erase/read* gate $e_h \in (0, 1)^N$ that scales the address under which the old content is retrieved, and a value-axis *write* gate $w_h \in (0, 1)^V$ that scales what is written (the GDN-2-inspired half of the design; Section 7′s split-gated theorem set pins the relationship). The display below is one head's full step in execution order: normalize the two addresses, gate the read address and the write value, read the memory, form the retrieval error, update the bounded state, and gate the read output. The formal core identities in Section 7 use abstract real-valued key/gate/state domains; they are not reachable-trajectory proofs for this production step.

$$
\begin{aligned}
k_h &\leftarrow \frac{\mathrm{silu}(k_h)}{\|\mathrm{silu}(k_h)\|_2} \quad (\mathrm{L}^2\text{ -normalized key}) \\
q_h &\leftarrow \frac{\mathrm{silu}(q_h)}{\|\mathrm{silu}(q_h)\|_2} \quad (\mathrm{L}^2\text{ -normalized query}) \\
e_h &= \sigma(E_h x_t) \quad \text{(key-axis erase/read gate)} \\
w_h &= \sigma(W_h x_t) \quad \text{(value-axis write gate)} \\
r_h &= S_h^T (e_h \odot k_h) \quad \text{(gated read at the key)} \\
\delta_h &= w_h \odot v_h - r_h \\
S'_h &= \tanh\big(d_h S_h + k_h \delta_h^T\big) \\
y_h &= \mathrm{silu}(g_h) \odot (S'_h)^T q_h
\end{aligned}
$$

The displayed normalization is idealized for nonzero addresses; production regularizes with `norm+1e-6` (shared fused normalization: `elman/models/e88_fused.py:239-242`).

Here silu is the smooth pointwise activation $x \odot \sigma(x)$, with σ the logistic sigmoid, v_h is the post-activation value input (the activated value supplied to the cell, so the write gate scales an already-activated write), and $\|\cdot\|_2$ is the Euclidean norm. The first two lines put the key and query on the unit sphere so the addressing is bounded; the middle lines make the write a correction against the content already read — under a gated address and at a gated magnitude — rather than a raw addition; and the final line gates only the read output. The state stays inside tanh, so the update is nonlinear-state for every gate setting. The formal specialization and adjacent-core identities are given in Section 7. This is the per-head primitive. Algorithm 1 spells out the same read-before-write order; its inputs are the projected quantities described above, and its output uses the returned state. It is not a full-stack implementation.

Algorithm 1 — E97 per-head step.

```
Input: S, projected k, q, activated v, d, e, w, g
1. k <- silu(k) / norm2(silu(k))
2. q <- silu(q) / norm2(silu(q))
3. r <- transpose(S) (e hadamard k)
4. delta <- (w hadamard v) - r
5. S_next <- tanh(d S + outer(k, delta))
6. y <- silu(g) hadamard (transpose(S_next) q)
Return: S_next, y
```

E97 repeats the same per-head equation across a residual stack.

E97 fixes these family choices to one billion-parameter instance: model width 1792, depth 11, $H = 216$ heads per layer, $N = V = 32$, expansion 1.0, a bias-free SwiGLU MLP of hidden width 4,032 after each mixer (mlp ratio 2.2623), per-rank batch size 4, a recurrence chunk length of 2048 tokens, and 1,286,589,072 parameters in total (1{,}048{,}152{,}720 non-MLP plus 238{,}436{,}352 in the eleven MLPs). For E97, at depth 11, $H = 216$, and batch size 4 the stack runs $11 \times 216 \times 4 = 9{,}504$ independent recurrent programs per token, each carrying a 32×32 state tile ($216 \times (32 \times 32)$ recurrent-state scalars per layer). Around the recurrent core the language-model wrapper is conventional — a pre-norm residual stack with a final normalization and tied input/output embeddings. Heads never share recurrent state; each layer learns its own key, query, value, decay, split-gate, and output-gate projections; and the recurrent state starts at zero when no prior state is carried in.

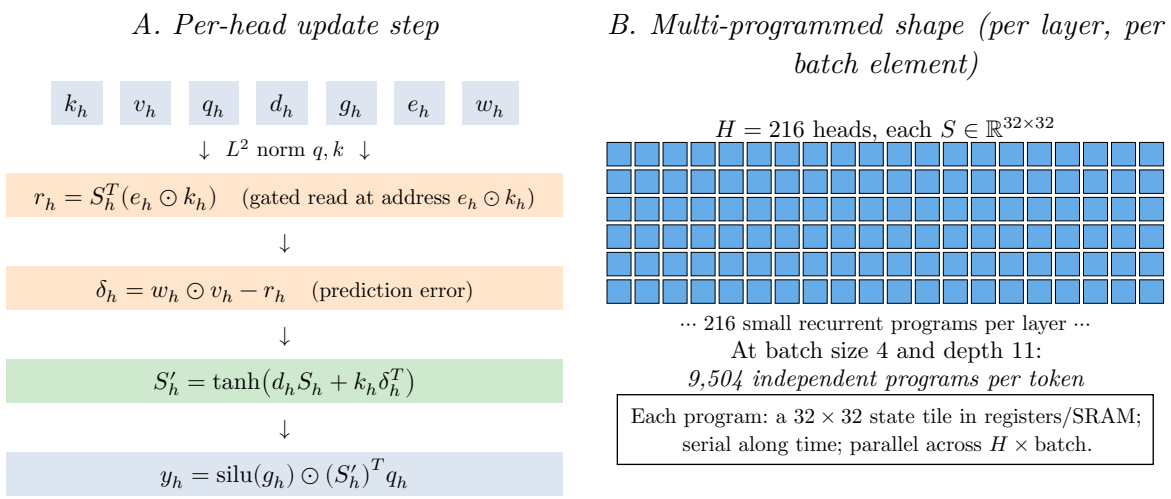


Figure 2: *Emender architecture and multi-programmed shape. (A)* Per-head recurrent dataflow (E97). Each head owns a small bounded associative matrix memory S_h. The model reads under the gated address $e_h \odot k_h$, computes a prediction error δ_h against the gated write value, writes the bounded delta correction into S_h, and gates the read at q_h for output. The tanh on the state, the delta-correcting write, the split erase/write gates, and the L^2-normalization of q, k are visible here; Section 3 enumerates the four load-bearing ingredients of the Emender family. *(B)* The 1.3 B stack exposes 216 small independent heads per layer per batch element. Each head is a 32×32 state tile that fits in registers/SRAM and runs its time loop serially; parallelism is harvested across heads, depth and batch (Section 4), giving $11 \times 216 \times 4 = 9\{,\}504$ programs per token at batch size 4.

For the Emender family, four ingredients are load-bearing; E97 uses all four in the instance above.

(a) *Bounded matrix state via* tanh *on* S. The state-channel duality (Section 7) distinguishes the executed nonlinear composition from an affine fold; placing the non-linearity on the state itself, not only on a gate or the output, is what makes an emender layer nonlinear-state.

(b) *Delta-correcting write* $w \odot v - S^T(e \odot k)$. The emender layer reads what memory predicts at the gated address $e \odot k$, computes the prediction error δ against the gated write value $w \odot v$, and writes the correction. Under unit decay and all-one gates, the pre-tanh core with orthonormal keys gives exact overwrite at one slot while preserving orthogonal slots (`linearDeltaWrite_overwrites_one_preserves_others`); full-state tanh, split gates, and arbitrary keys change that guarantee. The current pair shares delta correction; its distinction is the nonlinear carried-state path, together with the stated decay placement.

(c) *Saturation latching of the* tanh *bound.* A slot driven near ±1 resists perturbations under the saturation theorem's hypotheses; sign-preserving hold and counter-delta release have separate decay and write-budget assumptions. These motivate revisable memory rather than guarantee persistent bindings in production E97. Section 7′s Theorem set F (saturation insensitivity, sign-preserving hold, counter-delta release) formalizes all three as slot-wise statements on the full emender-layer update.

(d) *Many small heads, not one large matrix.* E97 at 1.3 B uses $H = 216$ heads of 32×32 each ($11 \times 216 \times 4 = 9\{,\}504$ independent programs per token at batch size 4). Per-head L^2-normalized q, k give many independent addressing programs rather than sharing one address path across the value heads.

GDN2-MLP state channel

Algorithm 2 defines the adjacent GDN-2 linear core used by the formal comparison: decay is applied *before* retrieval and correction, and no tanh is applied to the returned state. Its inputs are an already supplied key, split gates, value, and pre-decay matrix; it does not impose E97′s projection processing on GDN2-MLP. In the trained stack the mixer is followed by the SwiGLU MLP of Section 5, not by a state activation. This core specification does not assert that a multilayer stack has input-only gates. Algorithms 1 and 2 present the abstract core; the released baselines add projection, normalization, gate-domain, and residual machinery whose full-stack binding is carried by limitation G2 (Section 10).

Algorithm 2 — GDN-2 split-gated linear core.

```
Input: S, D, k, e, w, v
1. P <- D S
2. r <- transpose(P) (e hadamard k)
3. delta <- (w hadamard v) - r
4. S_next <- P + outer(k, delta)
Return: S_next
```

Parameterization choices

E97 parameterizes the per-head decay d_h in log space, following Mamba2 [2]: each head learns a scalar $A_{\log}$ and a bias δ_{bias} — the code names `A_log` and `dt_bias` — and forms the decay as $d = \exp\big(-\exp\big(A_{\log}\big) \cdot \text{softplus}(\alpha(x) + \delta_{\text{bias}})\big)$, where $\alpha(x)$ is the input-dependent term and σ is again the logistic sigmoid. Both decay parameters are exempt from weight decay, as is standard for such timescale parameters, so the regularizer does not pull the learned decay toward a default. The recurrence is computed in float32 and cast back to the storage precision, and the tanh on the state uses the numerically stable form $\tanh(z) = 2\sigma(2z) - 1$ in the fused kernel to avoid overflow at large pre-activations. Three remaining choices are: there is no output normalization inside the recurrent layer — an ablation removed it for a 0.10-nat gain, and the wrapper's block-level pre-norm plus final norm suffice — no short convolution on the input. E97 uses the split erase/read and write gates above; its output gate $\text{silu}(g)$ still acts on the read alone.

The 1.3 B stack

Ingredient (d) sets the 1.3 B stack's shape — many small heads rather than one large matrix — and the numbers follow from it: each head's 32×32 tile is small enough to live in registers (Section 4), the head count $H = 216$ is large enough to give that many independent addressing programs per layer, and eleven such layers — each followed by its SwiGLU MLP, the channel-mixer that pairs with the token-mixing recurrence — reach the 1.3 B-class budget the paired comparison fixes. The exact values were set by the per-architecture search of Section 5, not by hand. At batch size 4, depth 11 and $H = 216$ multiply to $11 \times 216 \times 4 = 9\{,\}504$ independent recurrent programs per token, each carrying its own 32×32 state tile. The reference implementation and the fused Triton recurrence kernel that steps these tiles are identified by the source register and release manifests.

The matched architectural contrast

E97 and GDN2-MLP both use matrix memory, delta correction, split editing gates, and a SwiGLU channel-mixing MLP. The key distinction for the present hypothesis is the operation

performed on carried state: E97 applies tanh after its correction, whereas the GDN2 state update is affine when its input-supplied parameters are fixed. Decay placement also differs, as the core identities above specify. The full networks contain nonlinear projections, gates, and MLPs, so this is not a comparison between a globally nonlinear network and a globally linear one. The matched-pair probes test what the trained architectures learn under the recorded protocol (Section 6); the formal analysis states the narrower update-map facts (Section 7).

4 Making sequential updates practical

Multi-programming: the throughput-enabling design choice

Width-axis multi-programming supplies throughput while each head's time loop remains serial. A linear recurrence earns its throughput on the *time axis*: because $S_t = A_t S_{t-1} + b_t$ unfolds into a product that depends on the inputs alone, the step-by-step loop can be re-expressed as a prefix scan or a chunkwise matrix multiplication over the whole sequence at once. A purely nonlinear recurrence cannot be rewritten that way without giving up the nonlinear update that makes it expressive, so its time loop has to run serially. *Multi-programming* recovers the throughput on the other axis. Rather than parallelize one recurrence along time, it replicates the recurrent computation across many independent heads — each with its own small bounded matrix state — and harvests parallelism across those heads, across state tiles, and across batch elements, while the time loop inside each head still runs one step at a time.

The design choice is a decoupling: it frees throughput from the *shape* of the update. The recurrent body must stay a bounded, register-resident per-step map at a fixed matrix-state signature — that is what keeps each head cheap — but it need not be compatible with an associative scan or a chunkwise WY product. The cost is paid in per-head serial time, most exposed at long sequence lengths where the serial loop is longest; the gain is that a nonlinear recurrence can keep a modern accelerator busy with no time-axis scan at all.

The 1.3 B Emender shape under multi-programming

A modern GPU exposes many independent streaming multiprocessors, each able to run a small program out of its own registers and shared memory, and multi-programming is built to fill them. E97, the 1.3 B Emender instance, runs 216 such programs per layer per batch element — one per head — so at depth 11 and batch size 4 the layer stack issues $11 \times 216 \times 4 = 9\{,\}504$ small independent recurrent programs per token, each carrying its own 32×32 state tile small enough to live in registers. Keeping the accelerator busy therefore does not require parallelism *along time*. The paired throughput and memory measurements below test the practical consequence at the actual E97/GDN2-MLP geometries; direct E97 occupancy profiling remains to be measured.

Measured throughput and utilization (CUDA only)

All throughput measurements and throughput-budget smokes reported here are CUDA-only; none establishes correctness, performance, or parity on ROCm.

At the actual 1.3 B pair geometries, the matched-batch preflight on seven RTX 6000 Ada GPUs under DDP measures E97-MLP at 3,211 tokens/s per GPU and GDN2-MLP at 3,290, both at batch 4. The ratio is 0.976: comparable throughput, with GDN2-MLP slightly faster. E97

uses 28,942 MB peak memory per GPU against 35,715 MB for GDN2-MLP. E97 therefore fits batch 6 under DDP, reaching 4,470 tokens/s per GPU, where the control cannot fit that batch (`experiments/preflight_100b/RESULTS.md`). The larger-batch throughput gain comes from the memory advantage, not a faster per-token kernel at the matched batch.

These are actual-geometry DDP preflight measurements. The long-run language-model training below uses eight independent local-DiLoCo islands, so these rates are not presented as its production throughput. Near-equal parameter counts and comparable measured speed do not establish identical actual FLOP counts; no paired measured-FLOP accounting is supplied here.

Fused Triton recurrence kernel

The forward kernel uses an internal `[T, B, H, *]` layout (time, batch, head, ...) and dispatches one program per `(batch, head_block)`. The `BLOCK_H` parameter is autotuned over $\{1, 2, 4, 8, 16\}$ because at $H \geq 256$ per-program-per-head launch overhead dominates. The state tile lives in registers and shared memory for the duration of the T-step loop. The following pieces are fused into the same Triton program: tanh/silu activations on input projections, L^2 normalization of q, k, the recurrent delta write, and the output gate. Each fusion removes one to two `torch.cuda` launches per layer call.

Sparse-checkpoint backward

The forward kernel saves S only every $K = 16$ steps. The backward kernel processes one K-step segment at a time: it forward-replays the K steps to rebuild per-step S_{t-1} from the saved checkpoint, then walks backward to apply the chain rule. The activation memory used by the kernel is $\frac{T}{K} + 1$ checkpoint slots instead of T. At $K = 16$ this is approximately a $16\times$ shrink, at the cost of a single extra forward pass per segment.

Triton source portability; CUDA-only validation

The kernel is written in Triton [25], a source-portability route rather than evidence of cross-backend parity. The pair's production runs, kernel throughput measurements, and expressivity throughput smokes all ran on NVIDIA CUDA. No ROCm smoke or paired production artifact supports a correctness or throughput claim here; no CUDA/ROCm parity is claimed. The successor E97 Frontier continuation is a separate, single-arm line of work, not a portability benchmark for the E97/GDN2-MLP pair.

Distributed training

The 1.3 B-class result reported here was produced on a single eight-GPU workstation, with no cluster and no sequence parallelism — both 150 B-token arms of Section 5 ran the same *local DiLoCo* recipe on that one machine: eight independent islands (one per GPU), schedule-free AdamW [26] inside each island, and plain model-weight averaging across islands every $\tau = 250$ local steps (outer learning rate 1.0, momentum 0 — the conservative plain-average form of DiLoCo [27]). Scaling the same recipe past one machine pairs schedule-free AdamW per island with hierarchical local-SGD averaging: each island is one node of 8 GPUs with intra-island DDP, and inter-island synchronization averages model weights on the same $\tau = 250$ cadence. The structural point holds either way: because the Emender's parallelism is across programs rather than along time, it does not need sequence parallelism to stay competitive at the 1.3 B-class

— a simplification relative to the chunked-scan implementations that linear-state recurrences rely on.

ParaRNN [6] offers a different systems route, using an iterative solver to parallelize nonlinear recurrence along time. The present implementation instead retains one serial pass per head and obtains its parallelism across heads and batch elements; no comparative ParaRNN performance experiment is claimed here.

5 Language modeling at 1.3 billion parameters

The systems design earns its place if the model can be trained on a substantial language-modeling workload. We compare E97 with a closely parameter-matched GDN2-MLP control under the same long-run recipe. The result is a narrow training-loss band at matched tokens. The following identities, alignment rules, and search records define that comparison.

The matched pair and its training recipe

The 1.3 B comparison is a *pairing*: one nonlinear-state model and one linear-state model, each selected by its own CMA-ES search, trained from scratch under a literally identical recipe, and read out at exactly matched aggregate tokens. The two arms are E97 — the Emender split-edit instance of Section 3, paired with a bias-free SwiGLU MLP per layer — and GDN2-MLP — the official-style Gated DeltaNet-2 [4] mixer with the *same* SwiGLU MLP, bound to external source commit `95709fc`. Holding the channel-mixer (MLP) fixed on both arms makes this a comparison of token mixers — the recurrence.

Training identity	*E97-MLP*	*GDN2-MLP*
Parameters	1,286,589,072	1,286,713,448 (+0.0097%)
Mixer	Emender split-edit delta (Section 3)	Gated DeltaNet-2, 4-token conv
Geometry	dim 1792, depth 11, $H = 216$, $N = V = 32$, mlp ratio 2.2623	dim 2176, depth 12, $H = 30$, expansion 1, mlp ratio 3.2587
Base LR (CMA-selected)	1.007×10^{-3}	4.743×10^{-4}
Data	The Pile, 2,048-token chunks, `p50k_base`	identical
Optimizer	schedule-free AdamW, bf16, per-rank batch 4	identical
DiLoCo	8 islands (one per GPU), $K = 250$, plain average, outer lr 1.0, $\beta = 0$	identical
Tokens per optimizer step	$8 \times 4 \times 2048 = 65\{,\}536$	identical
Run	`emender_E97_1.3B_20260709_084606`	`gdn2_gdn2-mlp_1.3B_20260722_083444`
Final step	2,300,930	2,323,616
Final aggregate tokens	150,793,748,480	152,280,498,176
Final last-100 loss (nats/token)	2.4365	2.4034
Source .pt checkpoint SHA-256	`0239706e1f67e4823008a3a2754894b5b94dc1663580d2e40c1c74f7dd6a72b2`	`ffae61cfeeeff820f469f7d66dd31c60e48c455872ff9102e57b15ad75bad59b`

Table 1: *Training identities of the 1.3 B pair.* Both arms ran the same launcher harness on the same eight-GPU workstation: same data file, chunk size, tokenizer, optimizer, DiLoCo policy, token cadence, and checkpoint cadence (saves every 500 steps, on merge boundaries); the arms differ only in mixer, geometry, and the CMA-selected learning rate. Sources: launch manifests `scripts/launch_emender_8gpu_diloco.sh` and `scripts/launch_gdn2_mlp_8gpu_diloco.sh`; per-run `args.json`/release validation receipts (`docs/EMENDER_DILOCO_OPS_HANDOFF_20260722.md`, `docs/validation/e97-huggingface-dense-base-and-cli-agent-release.md`, `docs/validation/gdn2-huggingface-152b-control-release.md`).

Both arms consumed 65,536 tokens per optimizer step (8 ranks × batch 4 × chunk 2,048, validated from the launch manifests and log deltas), so matched optimizer steps *are* matched aggregate tokens. GDN2-MLP was run 1.49 B tokens past E97′s final step so that the comparison could be read near E97′s terminal authority. The final common logged read-out is five optimizer steps earlier than that checkpoint; no paired loss statistic uses GDN2′s extra tokens.

Algorithm 3 separates selection, training, and comparison. It summarizes the recorded pair protocol rather than specifying a new search or training run; the selection account below identifies the retained winner configurations and the missing first-generation search records. The averaging window is reported, not selected to establish an ordering.

Algorithm 3 — pair selection, training, and loss alignment.

```
Input: per-family search bounds, candidate budget, Pile recipe
1. For each family, run CMA-ES with its candidate budget.
   Train candidates; score the recorded late-loss window.
   Retain the winner geometry and learning rate.
2. Train each winner on the common chunk/tokenizer recipe.
   Each island takes local schedule-free AdamW steps.
   Every 250 steps, average parameters (outer LR 1, beta 0).
   Record rank-0 losses and merge-boundary checkpoints.
3. Deduplicate each loss log by optimizer step.
4. Intersect logged step IDs through the common cutoff.
5. Compute raw and declared trailing-window loss differences.
Return: pair identities, aligned losses, windowed differences
```

Loss parity at matched tokens

The primary read-out is the training-log loss of both arms aligned on exact common logged steps through step 2,300,925 (150,793,420,800 tokens): 92,037 aligned records, zero E97-only and zero GDN2-only records through the cutoff, no interpolation. This is 327,680 tokens (five optimizer steps) before E97′s terminal checkpoint at step 2,300,930; the table is a last-common-log comparison, not an unlogged terminal-step measurement.

Statistic at the 150.79 B-token cutoff	*E97*	*GDN2-MLP*	*Δ (GDN2 − E97)*
Raw loss at cutoff step	2.4770	2.4324	−0.0446
80-point trailing smoothed	2.4370	2.4267	−0.0103
Mean, last 100 steps	2.4284	2.4423	+0.0139
Mean, last 1,000 steps	2.4358	2.4273	−0.0085
Early comparison, 5.118 B tokens (smoothed)	2.9300	2.8588	−0.0713

Table 2: *Matched-token loss comparison (nats/token, The Pile).* The common logged cutoff is step 2,300,925, not checkpoint step 2,300,930. At the matched cutoff the two arms sit within 0.010 nats/token smoothed (≈ 0.4% relative), and the *sign of the gap depends on the averaging window*: GDN2-MLP is lower on the full-trajectory and 1,000-step windows, E97 is lower on the most recent 100-step window. Early in training (5.1 B tokens) GDN2-MLP led by 0.071 nats; the gap closed as training progressed. Source: `docs/GDN2_OPS_SNAPSHOT_20260728.md` (final matched cutoff) and `docs/GDN2_E97_MATCHED_LOSS_COMPARISON_20260723.md` (early comparison); both parse rank-0 logs only, deduplicate by optimizer step, and align on exact common steps.

The two current models reach *comparable training loss* after each family's geometry and learning rate have been selected by search. The small, window-dependent difference does not establish a loss ordering or statistical equivalence. It establishes that the nonlinear recurrence trains competitively under this substantial shared workload. Under the run's recorded byte normalization (`bytes/token` = 3.783 from the frozen `args.json`), E97′s final-window loss of 2.4365 nats/token corresponds to an estimated *0.93 bits per byte* — sub-1-bpb territory for a purely nonlinear recurrence.

Two boundaries apply to this measurement. First, it is a *training-log* comparison: a held-out, tokenizer-invariant bpb measurement on the 150 B pair has not yet been run and is the pending evaluation item — TODO-NEEDS-EVIDENCE. Second, it is one run per arm, single-seed; the window-dependent sign of the Δ in Table 2 is the summary of a tie at this precision, not evidence

for either ordering. The carried-in winner configurations and the missing first-generation search records are described below.

Selection of the two models

Both arms were selected by CMA-ES with 15 minutes of Pile training per candidate at 2,048-token context, bf16, fused kernels, and a 1,300 M parameter target with ±3% tolerance. The family-specific search selected geometry and learning rate; no subsequent per-arm retuning was applied to either long run.

E97-MLP carries the emender-mlp winner: width 1792, 216 heads, depth 11, state size 32, MLP ratio 2.2623, and learning rate 1.007×10^{-3} (search fitness 5.8606). GDN2-MLP carries the `lb-gdn2-mlp` winner: width 2176, 30 heads, depth 12, expansion 1, MLP ratio 3.2587, and learning rate 4.743×10^{-4} (104 evaluations over 13 generations; best fitness 5.8949 at evaluation 86). The exact configurations and retained search records are in `experiments/lb_compare_20260613/REPRODUCTION.md` and `docs/repro/lb_gdn2_mlp_20260612/best.json`.

First-generation fitness is unavailable for these carried-in winner searches, and E97′s candidate/generation totals are absent from the normalized snapshot. Consequently, equal per-candidate rules do not establish equal total search effort, and within-search gains cannot be reconstructed for the winners. Open item U3.1 records this missing winner-series evidence; no broader search leaderboard is used as a result of the present pair.

Held-out language-modeling and downstream evaluations of the current pair remain to be run. They cannot be inferred from its training logs.

6 What the current models learn to compute

Comparable language-modeling loss leaves open what the carried state can learn to compute. We fine-tune the same 1.3 B E97 and GDN2-MLP checkpoints on two explicit computations. The modular-quadratic task repeatedly applies a nonlinear map to a running value; E97′s advantage supports the mechanism hypothesis that motivates its state update. The S_5 task composes finite-group actions; GDN2-MLP's strong result shows that this is a different learning problem, not a proxy for an all-purpose ordering of architectures. Both studies concern learned performance under stated budgets and protocols.

Iterating a nonlinear map with the trained pair

The decisive experiment uses the E97 and GDN2-MLP checkpoints of Section 5, with Schedule-Free y-mode recovery, on $x_t = (x_{t-1}^2 + c_t) \bmod 256$, 5 input drivers and running supervision. Both arms receive the identical full-fine-tune recipe: bf16 AdamW(betas=0.9,0.95), constant learning rate 0.00005, warmup 300, weight decay 0.01, gradient clip 0.5, batch 32, trained length 128, seed 42. The throughput-derived common cap of 18269 steps was fixed before accuracy was seen (`budget.json`, `1989656c`). It projects 4.35 GPU-hours of capped training, *not* actual expenditure. The same pre-registered validation-loss plateau rule stopped E97 after 2500 steps and GDN2-MLP after 2200 steps; actual training elapsed 0.649 GPU-hours, excluding loading and final evaluation. Thus this is a common-cap/common-stop-rule comparison, not equal realized training steps or tokens.

The protocol below summarizes the retained driver, `scripts/expressivity_1p3b_depthwall.py`, including its zero-based probe indices. It selects the *stop* weights, not the best observed

validation checkpoint. The terminal sweep is grad-free and scores every masked position using full-vocabulary argmax; reduced evaluation batches on OOM are recorded rather than interpreted as a new training recipe.

Algorithm 4 — matched-cap fine-tune and terminal evaluation.

```
Input: bound y-mode weights; common recipe, cap, seed
1. Build fixed validation batches; initialize AdamW.
2. For step j = 0,...,cap-1:
   a. Draw a batch at T=128; run bf16 forward.
   b. Backpropagate masked cross-entropy over all positions.
      On OOM, split the same batch and accumulate gradients.
   c. Warm up LR for 300 steps, then keep LR constant;
      clip gradient norm to 0.5; take an optimizer step.
   d. Every 100 updates (and at cap), probe validation loss.
      With 4 probes and j >= min(1000,cap), stop if
      loss <= 1e-4 OR both 200/300-step gains are < 1%.
   e. Every 250 updates, record trajectory probes unless stopped.
3. Retain stop weights and actual update count for each arm.
4. For T in {128,256,512,1024,2048,4096,8192}:
   a. Set B=max(1,floor(32768/T)); build 8 seeded batches.
   b. Compute masked accuracy and loss without gradients.
      On OOM, halve B and rebuild with the same length seed.
      If B=1 fails, record an error rather than a score.
Return: training record, stop reason, terminal length sweep
```

Relative gain divides the loss decrease by the earlier loss magnitude (floored at 10^{-12}); both trailing windows must pass. The implemented minimum uses the zero-based index j, so 1,000 updates alone do not trigger it. The cap, stopping criterion and recipe are common; realized training lengths in updates are not equal.

Length	*Train-length multiple*	*E97*	*GDN2-MLP*
128	1×	99.96%	76.09%
256	2×	99.14%	62.85%
512	4×	90.16%	56.55%
1024	8×	76.07%	53.36%
2048	16×	62.92%	51.91%
4096	32×	55.79%	50.74%
8192	64×	51.79%	50.53%

Table 3: *Modular-quadratic length extrapolation on the released 1.3 B pair.* Full-vocabulary argmax accuracy at the pre-registered plateau stops; trained length 128, seed 42, 262144 scored token positions per length. The identical cap is 18269 steps, not identical realized training: E97 stopped after 2500 steps and GDN2-MLP after 2200. E97 decays gracefully rather than remaining length-invariant; GDN2-MLP approaches a roughly 50% plateau, *not* the declared 0.39% random baseline. Source: `paper/results/expressivity_1p3b/iterated-map/`, `ft_{e97,gdn2}_modular_quadratic{,_sweep}.json` (commit `9dc3afd7`); `budget.json` (pre-registration `1989656c`).

At the terminal sweep E97 reaches 99.96% at trained length, retaining 90.16% at four times and 76.07% at eight times that length. GDN2-MLP ends at 76.09% at trained length and 53.36% at eight times. E97 at eight times approximately matches GDN2-MLP's terminal trained-length accuracy, then decays further; it is not length-invariant. GDN2-MLP approaches a roughly 50%

plateau at the longest lengths, above the declared 0.39% random baseline. The terminal trained-length value is not a hard ceiling: intermediate GDN2-MLP trajectory probes were higher. One seed, unequal plateau-stop lengths, and validation fluctuations limit the inference; this is the measured modular-quadratic separation under this protocol, not a theorem that the full stack cannot learn the task.

The modular-quadratic finding is consistent with the temporal-composition contrast proved in Section 7: E97 executes one nonlinear state update per step, whereas a GDN fold conditioned on an input-only schedule remains affine in its initial state. These are state-channel facts, not an impossibility or a minimal-depth bound for the full GDN2-MLP stack. The S_5 routing result further assumes a token-only schedule and a decoder separating reachable states; it locates the required token-dependent actions without ruling out affine group representations. The group-state experiment below examines a computation for which that distinction matters.

S5 group-state tracking on the same pair

The S_5 word problem tests repeated composition of permutation actions. It is structurally distinct from the modular-quadratic map: affinity of a conditioned state channel alone does not prohibit a finite-group action. The current trained pair therefore provides a useful second test of what these architectures can learn efficiently.

Zero-shot evaluation did not separate the two models. Matched fine-tuning favored E97 on the solvable S_3 control. In the longer-horizon S_5 capacity study, GDN2-MLP is ahead of E97 at every reported length, and both curves decline toward the random baseline. This is a strong GDN2-MLP result on group-state tracking. It supports discussing the task's learned performance and learning efficiency within the architectures' capabilities, rather than treating the result as a universal state-channel impossibility or an architectural expressivity ranking. Sources: `paper/results/expressivity_1p3b/SUMMARY.md` (`5e1e60ef`) and `capacity/SUMMARY.md` (`6f916843`).

Length	*E97*	*GDN2-MLP*
256	0.397	0.685
512*	0.204	0.363
1024	0.109	0.190
2048	0.061	0.103
4096	0.033	0.054
8192	0.021	0.034

Table 4: *S5 group-state tracking on the current 1.3 B pair.* Full-vocabulary argmax accuracy, seed 42 both curves decay and GDN2-MLP stays ahead. *GDN2-MLP completed one optimizer step at length 512 before the curriculum stage proved OOM-infeasible even at sub-batch 1; this point is not wholly untrained. Source: `capacity/cap_{e97,gdn2}_s5_sweep.json` and `cap_gdn2_s5_train.json` under `paper/results/expressivity_1p3b/` (E97 `f1ed5615`, GDN2 salvage `6f916843`).

Both capacity curves decay rather than learn length-invariant tracking; held-out transitions remain weaker than seen ones. GDN2-MLP's attempted length-512 curriculum stage was OOM-infeasible even at sub-batch 1, after one completed optimizer step. This hardware limitation and the single seed constrain the result. Neither the bridge nor the duality supplies an impossibility theorem for the full GDN2-MLP stack. The measured advantage on S_5 and the measured E97

advantage on the iterated map answer distinct questions about learned computation under these protocols.

Interpreting the current-pair evidence

The iterated-map result is the direct empirical test of the motivating mechanism: E97′s nonlinear state path repeatedly transforms the value that will enter the next update. Its measured advantage is consistent with that hypothesis, while optimization, geometry, and checkpoint selection remain possible contributors. The S_5 result demonstrates that GDN2-MLP learns a different carried-state computation strongly. Together with comparable language-modeling loss, these measurements make the useful question concrete: which computations benefit from placing the nonlinearity directly in the temporal state path, and under what training and resource conditions?

7 What the update rules establish

The experiments measure learned behavior. The formal results describe the update maps and the resource classes in which they are compared. The first result below states the temporal composition that motivates the paper: E97 repeatedly applies a nonlinear state map, while a GDN state channel conditioned on a fixed schedule is affine in its initial state. The remaining results describe the finite-state task, the adjacent split-gated cores, and conditional latching.

Each set retains its exact hypotheses and source theorem names. These statements do not identify the minimum depth of an alternative representation or predict what SGD will recover. The distinction is especially important for the S_5 results: a generator-table lookup does not realize the full recurrent tracker, and affinity alone does not prohibit a group representation.

Theorem set H: compositional-depth duality and routing

The concrete repeated-map family is the schedule of `CompositionalDepthDuality.E97Step` values (k, e, w, v) at fixed λ (Lean field `b` is the paper's erase gate e), acting through `e97Step` on matrix state. `e97Step_eq_tanh_comp` identifies the step as elementwise tanh after the state-affine split-gated core. `e97StateAfter_eq_compose` gives its time-ordered composition, and `e97_compositional_depth_grows_in_time` states that appending a step applies one more such map. Thus the *executed* composition contains one tanh per token. This is not a proof that the resulting function has no shallower representation.

For a fixed, state-independent schedule of `CompositionalDepthDuality.GdnStepParams`, `gdn_state_channel_depth_bounded_in_time` gives matrices A, C with $S_T = AS_0 + C$ for every initial state (Lean writes H for this matrix, not the head count). The nonlinear temporal depth of that *state channel* is zero; a multilayer GDN2-MLP stack, state-dependent gates, approximation error, and alternative encodings are not covered.

`S5GDNBridge.gdn_s5_realization_requires_per_token_action` assumes a token-only schedule and a decoder that recovers `S5Tracker.run` on every word and separates reachable states. It then gives a state map F through the group element, with affine word actions implementing right multiplication on the reachable image. `gdn_s5_constant_parameters_cannot_realize` rules out a token-independent schedule. The bridge locates token-dependent actions; it does not rule out affine group representations or lower-bound the network generating their parameters. G1–G3 (Section 10) record those gaps.

Theorem set A: finite-state ceiling and S_5 tracker

At fixed width and precision the Emender is a finite-state machine, and the S_5 word problem it is tested on is a concrete, fully specified finite-state target.

- *Finite-state ceiling at fixed precision.* At fixed width and precision, every online recognizer of this kind has only finitely many distinct states (`S5Witness.fixed_precision_state_space_finite`). This bounds its online memory to finitely many states; it does not prove strict containment below NC[1].
- S_5 *word problem.* The target is pinned down exactly in Lean. The group S_5 has 120 elements (`S5Witness.s5_state_count`) and is non-solvable (`S5Witness.s5_not_solvable`); the prefix-product tracker that reads a word of transpositions and reports the running product is a 120-state recognizer (`S5Tracker.recognizer_state_count`) whose state composes correctly as tokens are appended (`S5Tracker.run_append`). A bridge lemma confirms the Lean tracker agrees with the Python evaluation harness on every input (`S5Tracker.pythonRun_eq_tracker_tuple`). The four adjacent transpositions generate all of S_5: the tracker's run map is surjective (`S5Inseparability.run_surjective`, via Mathlib's `Equiv.Perm.mclosure_swap_castSucc_succ`).
- *Capacity lower bound (recognizer floor).* Any finite-state recognizer with a state decoder that reproduces the tracker on every input word needs at least 120 states (`S5Inseparability.capacity_lower_bound_120`); equivalently, no recognizer with fewer than 120 states can realize it under any decoder (`S5Inseparability.small_recognizer_cannot_realize_s5`). The bound has a distinguishing-pair form: any recognizer with too few states admits two words it cannot tell apart that the tracker does distinguish (`S5Inseparability.exists_distinguishing_word_pair`). This is the existential form of the S_5-coset inseparability target: it holds for *any* finite recognizer family, with no architectural assumptions. The floor alone does not settle constrained realization by either of the trained architectures.
- *Lookup-table realization.* The tracker is small enough to write down as a table: it uses exactly $120 \times 4 = 480$ state/input keys (`S5NDMRealization.s5_transition_key_count`), and any finite recognizer admits such an exact lookup-table realization (`S5NDMRealization.exactTransitionMemory_run`).

Theorem set B: generator-table lookup, not a recurrent tracker

The exact statement of `NDMRealizesS5.emender_realizes_s5_tracker` supplies $d = 12$, four orthonormal keys and one-hot values, queries, a decoder, and $\lambda = 1$. Reading a *preloaded* table retrieves the current generator; decoding it and *externally multiplying a supplied* S_5 state s reproduces the transition $s \cdot g$. The linear readout retrieves a generator value, not the running product. The supporting `OnlineMemory.linearDeltaWrite_overwrites_one_preserves_others` is an exact *pre*-tanh overwrite statement. Neither theorem states that iterating the nonlinear cell from zero tracks every input word.

Open item O1 — recurrent tanh*-tracker realization unavailable.* The loaded generator memory does not carry the running group product; external composition supplies the missing state computation. Closing O1 requires a configuration of the actual nonlinear update, an explicit

initial state and decoder, and an all-words trajectory theorem. This is not a missing proof of the already-proved lookup identity, but a stronger, currently unproved architecture claim.

Split-gated E97 and GDN-2 cores

The module `SplitGatedDelta` identifies adjacent split-gated linear cores with different decay placement. The identities are over abstract real-valued gate/key/state signatures; they are not a whole-stack production inclusion theorem for the trained pair.

- *Adjacent split-gated cores with different decay placement.* The GDN-2 linear core is the E97 linear core applied to a pre-decayed state (`gdn2LinearCore_eq_e97LinearCore_on_decayed_state`), and under identity decay the two coincide (`gdn2LinearCore_identity_decay_eq_e97LinearCore_one`); the resource signatures then split exactly on the axis the paper tests empirically: E97 has matrix state, full-state temporal nonlinearity, delta writes, and no scan-compatible state path (`e97_split_gated_resource_signature_like_e88`), while the GDN-2-style split erase/write resource has matrix state and delta-style writes but *no* temporal state nonlinearity and a scan-compatible linear state (`gdn2_split_erase_write_is_scan_compatible_linear_state`).

The current pair therefore shares adjacent split-gated delta cores and differs on the nonlinear state path and decay placement, as well as geometry and input processing in the trained stacks. This identifies the formal structural distinction relevant to the iterated-map hypothesis. It does not isolate every contributor to the trained models' accuracy difference.

Theorem set F: latching

Set F establishes conditional slot-level saturation, hold, and release bounds, not production-default persistent binding.

These three results formalize the latching half of the Emender primitive: saturation makes a slot insensitive to further bounded writes, sign is preserved under sub-threshold counter-input, and a sufficient counter-delta releases the latch. All three are slot-wise statements on the full abstract Emender update, where v denotes the bounded post-nonlinearity value input already supplied to the recurrent body, $S' = \tanh\Big(\lambda S + k(v - S^T k)^T\Big)$ at slot (i, j), with $\omega := k_i\Big(v_j - (S^T k)_j\Big)$ the write contribution into that slot. The proofs run through the standard tanh-perturbation foundation. The module carrying them (`EmenderLatching`, introduced at `696299b4`) is included in the refreshed inventory and the current clean `lake build ElmanProofs.PaperCore`.

- *Saturation insensitivity.* If the slot is saturated in the sense $|S'_{i,j}| > 1 - \varepsilon$, and a perturbation δ to the pre-activation entry has $|\delta| \le M$ with $\varepsilon + M < 1$, then the post-tanh slot moves by at most $2(\varepsilon + M)\ |\delta|$. The bound shrinks linearly in both the saturation slack ε and the perturbation magnitude M; at deep saturation a bounded write barely moves the slot. Lean: `EmenderLatching.emender_saturation_insensitivity`. A two-input corollary `EmenderLatching.emender_saturation_two_inputs_close` lifts the bound to two value vectors sharing one key.
- *Sign-preserving hold.* With $\lambda \ge 1$ and a bounded write $|\omega| \le T < \lambda(1 - \eta)$ at slot (i, j), a slot already saturated with $|S_{i,j}| > 1 - \eta$ keeps its sign after one Emender step and stays bounded below in magnitude by $\tanh(\lambda(1 - \eta) - T)$. For $\lambda > 1$ and small η, T the argument $\lambda(1 - \eta) - T$ exceeds 1, so the lower bound exceeds

$\tanh 1 \approx 0.76$ and the latch holds with margin. The hypothesis $T < \lambda(1-\eta)$ is the "no sufficient counter-delta" clause; counter-delta release is the next result. Lean: `EmenderLatching.emender_latch_holds_by_default`.

- *Counter-delta release.* With $\lambda > 0$, $S_{i,j} > 0$, and a write contribution $\omega < -\lambda S_{i,j}$ in the opposite sign, the post-update slot is strictly negative — the latch flips out of its basin in one step. Symmetrically for the negative basin. The release threshold is $T^* := \lambda\,|S_{i,j}|$; any counter-delta exceeding T^* in the opposite sign suffices. Lean: `EmenderLatching.emender_latch_releases_on_counter_delta`. The quantitative companion `EmenderLatching.emender_latch_release_quantitative` shows that if the counter-delta exceeds the threshold by margin $\mu > 0$, the post-update slot lies on the opposite side of the origin with magnitude at least $\tanh \mu$.

Complexity statements and explicit non-claims

Lean-proved: finite-state memory at fixed precision (`S5Witness.fixed_precision_state_space_finite`), a non-solvable 120-element group (`s5_not_solvable`, `s5_state_count`), the exact abstract tracker, the recognizer floor (`S5Inseparability.capacity_lower_bound_120`), and the generator-table lookup of set B. Full recurrent tanh-tracker realization is O1, not a theorem. *Cited, not formalized here:* Barrington's NC^1 completeness result [24] and circuit containments for restricted SSMs and transformers under their respective arithmetic assumptions [18], [19], [20]. No strict separation of TC^0 and NC^1 is known or assumed here.

The core proves neither a families-wide linear-scan S_5 impossibility, Barrington's theorem, a per-token circuit-size/depth lower bound, learned weight recovery of a recurrent tracker, or robust finite-precision tracking by the trained models. Latching also remains conditional: the hold theorem assumes $\lambda \geq 1$, whereas production decay is below one, so it is not a default persistence guarantee for the trained E97. These are the boundaries that the empirical and applications sections retain.

Verification of the formal artifact

A trusted Lean 4 [28] core built on Mathlib [29] carries the formal spine: each result is stated as a theorem and compiled together with a proof of it. The import closure of the `ElmanProofs.PaperCore` module — 17 project source files in the recursive local closure — uses none of Lean's escape hatches: no `sorry`, `admit`, added `axiom`, `opaque` declarations, or `native_decide`. The gate is `formal/lean/scripts/check_paper_core.sh`; compiling `ElmanProofs.PaperCore` additionally checks elaboration of the proofs. The refreshed machine-extracted inventory is committed at `paper/data/lean-trust-gate-theorems.json` (857 named declarations, 723 user-written, 17 files; extraction source commit `e58ae937`), including the split-gated, latching, duality, routing, Lipschitz, and E1H-definition modules. Both the gate and extractor traverse the recursive local imports rather than a selected module allowlist. This check covers project sources, not the axioms or source files of Lean and Mathlib dependencies.

8 Repeated nonlinear maps in the wild

The iterated-map probes ask a question that arises well beyond language modeling: can an architecture keep applying a nonlinear transformation to its own evolving state? Consider the broad family

$$S_{t+1} = F_{\theta(S_t, u_t)},$$

where θ is fixed across steps, u_t injects the current input, and $F_{\theta(\cdot,u)}$ is generally non-affine in the carried state. A common form is $S_{t+1} = \varphi(A(u_t)S_t + b(u_t))$; neither an additive injection nor a smooth φ is required. Fixed θ describes the update law, not identical token-dependent coefficients: in the Lean schedule, each step supplies its own key, gates, and value. The formal E97 fold fixes λ; the architecture additionally supplies input-dependent decay. Autonomous maps have no input injection. The Emender's coupled update

$$S' = \tanh\Big(\lambda S + k(v - S^T k)^T\Big)$$

is an instance with state S, input (λ, k, v), and a fixed delta-correcting tanh update; it is the all-one split-gate specialization of E97 (Section 3). This is a structural family, not a claim that the systems below share E97's weights, learning rule, or computational difficulty. Algorithm 5 is its time-ordered execution: each returned state is the next step's input. For the abstract E97 fold, the schedule supplies (k, e, w, v) at fixed λ; for GDN it additionally supplies D and is independent of the carried state. The application displays below specify F_θ and the state/input coordinates for each instance of this same loop.

Algorithm 5 — repeated-map fold.

```
Input: initial state S_0, fixed update law F_theta, input schedule
1. S <- S_0
2. For each input u in time order:
   S <- F_theta(S, u)
Return: S
```

The depth-duality characterizes executed state-channel composition; application-level lower bounds remain open. `CompositionalDepthDuality.e97_compositional_depth_grows_in_time` records one additional nonlinear update per time step; `gdn_state_channel_depth_bounded_in_time` proves that a GDN state fold is affine in its initial state *conditional on a state-independent input/gate/value schedule*. Neither proves that the resulting function has no shallower representation in other coordinates. The S_5 bridge (`S5GDNBridge.gdn_s5_realization_requires_per_token_action`) is a routing statement under its reachable-state-separating decoder hypothesis, not a depth lower bound. It does not transfer a separation to the applications below; group actions can admit affine representations. Here *EXHIBIT* means a definitional or structural instance of the family, *ANALOGY* a related but non-identical mechanism, and *OPEN* a prediction not yet established. In each case the question is whether a fixed-depth GDN-style linear-state approximator can reproduce the required trajectories, not whether the original engineered system already works.

Sigma–delta conversion: quantization inside the loop

EXHIBIT. A first-order sigma–delta modulator accumulates an analog sample and feeds back a quantized output [30]:

$$a_t = r_t + x_t, \quad q_t = Q(a_t), \quad r_{t+1} = a_t - q_t.$$

Here r_t is the integrator residue, x_t the injected sample, and the fixed state map is $F(r, x) = r + x - Q(r + x)$. For a one-bit converter, $Q(a)$ selects one of two output levels: a hard, discontinuous nonlinearity rather than E97′s smooth saturation. The nonlinear result is fed back into the next state, not merely placed in an output readout. Such streaming error-feedback loops are implemented directly in converter circuitry; they need no large learned processor to execute the update. *OPEN:* the duality suggests testing a GDN surrogate near quantizer thresholds over long streams, where residual errors can change later bits. A nonlinear state update represents this feedback directly; a linear residue integrator with an output-only quantizer omits the feedback dependence. No GDN lower bound follows: finite-resolution states, alternative encodings, or a different error metric may make a linear-state surrogate adequate. Predicting filtered signal quality is also weaker than predicting the exact bitstream.

Adaptive filtering and automatic gain control

ANALOGY for ordinary normalized LMS; EXHIBIT for the nonlinear variants. For regressor x_t, desired sample d_t, and coefficient state w_t, normalized LMS [31] uses

$$e_t = d_t - w_t^T x_t, \quad w_{t+1} = w_t + \mu x_t \frac{e_t}{\varepsilon + \|x_t\|^2}.$$

With inputs fixed this is *affine* in w_t: normalization alone is not a nonlinear-state example. A componentwise coefficient-limited variant, $w_{t+1} = \mathrm{clip}_{B\left(w_t + \mu x_t \frac{e_t}{\varepsilon + \|x_t\|^2}\right)}$, is one: state w, map the normalized correction followed by clipping, injection (x, d). The clipping is an additional engineering choice, not part of standard normalized LMS. Automatic gain control similarly feeds the measured output level back into the gain [32]. A representative bounded log-gain controller is

$$y_t = \exp(g_t) x_t, \quad g_{t+1} = \mathrm{clip}_{[g_{\min}, g_{\max}]}(g_t + \mu(P_0 - y_t^2)).$$

Its state is g_t, its injection is the sample x_t and target power P_0, and its fixed map includes the state-dependent power measurement and gain limiter. These equations specify representative variants, not universal implementations. *OPEN:* the duality predicts a possible surrogate gap when repeated limiter crossings or gain changes retain consequential history. Ordinary normalized LMS needs no such gap, and a stable AGC loop may forget errors rapidly enough for a linear-state approximation to suffice. These are natural per-sample edge and sensor workloads, but no IoT deployment advantage is measured here.

Proximal iterations and unrolled optimization

EXHIBIT. For the objective $\frac{1}{2}\|Az - b\|^2 + \rho\|z\|_1$, iterative soft-thresholding (ISTA) [33] repeatedly applies

$$z_{j+1} = \mathrm{soft}_{\eta\rho}\big((I - \eta A^T A) z_j + \eta A^T b\big),$$

where $\mathrm{soft}_{\tau(a)} = \mathrm{sign}(a)\max(|a| - \tau, 0)$ acts componentwise. The iterate z_j is the state, the fixed map is a gradient step followed by soft-thresholding, and $\eta A^T b$ injects the problem data at every iteration. Here time means optimization iteration, not arriving samples. FISTA [34] adds extrapolation,

$$z_{j+1} = \mathrm{soft}_{\eta\rho}\big(y_j - \eta A^{T(Ay_j - b)}\big), \quad y_{j+1} = z_{j+1} + \beta_{j(z_{j+1} - z_j)}.$$

Its state must include the previous iterate and momentum schedule (or its clock); it is not a single fixed map on z_j alone. Unrolling these updates exposes their nonlinear depth explicitly; changing weights at every unrolled stage forfeits the fixed-map identification. *OPEN:* the

duality predicts difficulty for a fixed-depth linear-state surrogate when support changes must be recomputed over many iterations. It does not say ISTA always requires growing nonlinear depth: after support and signs stabilize, soft-thresholding is affine on the visited region, and convergence can make a bounded-accuracy shortcut sufficient.

Reservoirs and echo-state networks

EXHIBIT. The standard driven echo-state network [35] has

$$h_{t+1} = \tanh(Wh_t + Uu_t + b).$$

The reservoir state is h, the fixed recurrent map is tanh after mixing by W, and the injection is $Uu_t + b$. With recurrent parameters held fixed, E97 likewise defines a driven nonlinear dynamical reservoir: its matrix state is mixed, corrected by the input, and passed through tanh again at each step. *ANALOGY:* it is not the conventional fixed-W echo-state architecture or training protocol. E97′s effective state mixing depends on the input key and gates, its recurrent weights are trained, and bounded tanh state alone does not establish the echo-state property (forgetting the initial condition). *OPEN:* for a reservoir task with persistent nonlinear history the duality motivates comparing E97 with a linear-state surrogate across length. A contractive reservoir instead has fading memory and may admit a good short-history approximation. The structural match does not predict a universal advantage over linear reservoirs with nonlinear readouts, nor establish that E97 satisfies the echo-state property.

Discrete ecology: the logistic map

EXHIBIT. May's population-dynamics example [36] is the autonomous logistic map

$$n_{t+1} = rn_{t(1-n_t)}.$$

State is normalized population n_t; r is a fixed model parameter; $F_{r(n)} = rn(1-n)$ is applied to its own output with *no* input injection. An environmentally driven variant can supply r_t as the input, but is then an extension of the autonomous model. This is the same logistic family as the contractive control in Section 6, not a claim that its parameter regime represents all ecological dynamics. *OPEN:* the duality motivates a long-horizon surrogate comparison only when nonlinear dependence on prior state remains relevant. The control is a countercase: contraction suppresses trajectory errors and no depth-wall signature is observed. Periodic attraction can also simplify prediction; chaotic regimes can make precise long-horizon prediction difficult for *both* architectures. Nonlinearity alone therefore cannot imply a separation, and an ecological population equation is not evidence of a deployed forecasting improvement.

Feedback control with limited actuators

EXHIBIT. A discrete-time linear plant with a smooth actuator limit and state feedback can be written

$$a_t = a_{\max} \tanh\left(\frac{Kx_t + Lr_t}{a_{\max}}\right), \quad x_{t+1} = Ax_t + Ba_t + Ed_t.$$

Here x_t is plant state; reference r_t and disturbance d_t are the injections; the fixed closed-loop map is $F(x,(r,d)) = Ax + Ba_{\max}\tanh\left(\frac{Kx+Lr}{a_{\max}}\right) + Ed$. Saturation in feedback is a standard nonlinear-control concern [37]; tanh is a smooth bounded-actuator model, not a claim that all actuators implement it. *ANALOGY:* unlike E97, the nonlinearity acts on the control path and the plant state need not be bounded. *OPEN:* the duality predicts a possible linear-state-surrogate deficit when state-dependent saturation repeatedly determines subsequent control

decisions. It makes no prediction of a gap in the unsaturated, approximately linear regime, or for a sufficiently contracting closed loop. A learned surrogate would additionally need stability and safety validation; a state-tracking result is not a control guarantee.

These examples make the architecture question concrete. When a workload requires persistent, repeated nonlinear transformation of its carried state, the duality says that nonlinear *state computation* cannot simply be dropped while leaving the same update in the same coordinates: an affine state channel does not perform that operation. In that restricted sense nonlinear-state recurrence is not optional; an alternative must supply an adequate representation or approximation elsewhere. Whether a fixed-depth GDN-style stack can do so at acceptable width, precision, latency, and horizon remains *OPEN* for these applications. Stable, fading-memory and affine-representable cases are explicit exceptions to any blanket advantage claim. The practical opportunity is many small, low-cost, low-intelligence streaming processors operating in parallel at the edge: local feedback and correction. Width-axis multi-programming suggests how many such nonlinear programs can coexist without linearizing their internal time loops; low-power silicon, energy savings, and task-level superiority would require separate measurements.

9 Related Work

Ancestry: the fast-weight line on delta correction

The Emender update belongs to a long line that treats memory as a structure to correct rather than to refill. The earliest formulation is Widrow and Hoff's least-mean-square rule [38], which writes the error between target and current output back into the weight vector of an adaptive linear element. Schmidhuber recast the same idea in a recurrent setting under the name *fast-weight programmers* [39]: a slow controller emits keys and values that update a fast-weight matrix online, with the update being a correction against what the matrix currently predicts at the addressed slot. Schlag, Irie, and Schmidhuber made the bridge to modern attention explicit [40], showing that linear-transformer kernels with cumulative outer-product updates *are* fast-weight programmers with a raw additive write, and proposing the delta-rule write as a stability and capacity fix. DeltaNet [14] carried the delta rule into a parallelizable linear-recurrent language model, demonstrating the rule at billion-parameter scale in the linear-state regime. The Emender extends this line with three properties the linear instantiations forfeit: a nonlinear matrix state, the saturation latching of Section 3 ingredient (c), which motivates persistent binding under the conditional slot-level hold result ($\lambda \geq 1$ plus a write budget), not an achieved production-default property; and width-axis parallelism via multi-programming that recovers throughput without time-axis linearization.

Linear-state recurrent language models

The dominant comparison cohort is the family of linear-state billion-parameter recurrent LMs surveyed in Section 2: Mamba [1] and Mamba2 [2] (selective SSMs); RetNet [41] (decayed linear attention); GLA [13] (gated linear attention); DeltaNet [14] (delta rule, linear in S); Gated DeltaNet [3] (gated delta rule); RWKV-4/5/6/7 [42], [43] (linear-state WKV / generalized delta rule with linear state); HGRN2 [44] (gated linear RNN with state expansion); MinGRU/ MinLSTM [16] (input-only gates, linear scan); mLSTM/xLSTM-7B [45] (covariance update); Griffin/RecurrentGemma [17] (RG-LRU, often hybridized with local attention). Circuit-containment results for restricted SSM families require their stated transition and arithmetic

assumptions [18]; affinity alone is not a universal complexity classification. The empirical panel in Section 6 directly tests Gated DeltaNet as the matched representative of this cohort and observes S_5 collapse at length.

In the broader SSM/sequence-model lineage, S5 [46] introduces simplified state-space layers with a multi-input multi-output linear state-space recurrence. It is distinct from the permutation group S_5 used as a probe here; the cited S5 results concern sequence models, not billion-parameter recurrent LMs.

A newer linear-state result, Gated DeltaNet-2 [4], decouples channel-wise erase and write gates and is reported by its authors to outperform GDN. It extends retrieval and state-tracking behavior inside the linear-state envelope, and it is the linear-recurrent baseline of this paper's 1.3 B pairing: the GDN2-MLP arm of Section 5 is the official-style Gated DeltaNet-2 mixer (external source commit `95709fc`) with the same SwiGLU MLP as E97, selected by the same CMA-ES protocol. The E97 split gates of Section 3 are the Emender's corresponding edit — key-axis erase/read plus value-axis write — inside a *nonlinear* state map, so the two architectures are adjacent by design (Section 7). Its decoupled gates are a natural next ablation for the multi-programmed substrate, being the kind of bounded per-step-body variant that does not require a new time-parallel scan.

Pure-nonlinear-recurrent peers and adjacent nonlinear-state work

M²RNN [5] (Mishra, Tan, Stoica, Gonzalez, Dao 2026) introduces nonlinear matrix-state recurrence with a raw-write update $Z = \tanh(HW + kv^T)$. It shares the central systems premise of this work — that a nonlinear recurrence is serial in time and must take its parallelism from the batch and head axes — and demonstrates that the update trains at 410 M dense and 7 B MoE scale on Nemotron-CC-v2. Its headline demonstrations use hybrid stacks that interleave attention and nonlinear recurrence. The current study instead evaluates the pure recurrent E97 mixer against a matched GDN2-MLP control. It provides related architectural context rather than an additional empirical comparator in this study.

xLSTM-1.3B [21] is a 7:1 mixture of mLSTM (linear) and sLSTM (nonlinear) blocks; with 87.5% of its blocks linear-state, it sits closer to the linear-state class than to the pure-nonlinear-recurrent one. It is the closest scale band among prior nonlinear-recurrent results, included here as a peer with a small nonlinear-block share. The xLSTM-7B follow-up [45] uses *only* mLSTM (linear) blocks, further from the pure-nonlinear-recurrent class still.

Titans [22] uses an MLP memory with online gradient updates and is qualitatively nonlinear-state, but it interleaves attention layers and has not been evaluated as a pure recurrent model at Pile-class scale. *Zeroth-order LSTM scaling* [47] demonstrates 1 B-scale LSTM training under a non-gradient regime, outside the standard-gradient comparison drawn here.

Classical LSTM/GRU [12] is the historical background: no published classical LSTM/GRU model has reached ≥ 500 M parameters on a Pile-class corpus. It is the unsuccessful empirical record that motivated the field's move to linear-state and hybrid alternatives. This is the assumption this paper revisits.

10 Limitations

Formal gaps: G1–G3 and O1

G1 — no per-token circuit-size or depth lower bound. `S5GDNBridge.gdn_s5_realization_requires_per_token_action` routes a hypothesized realizer's affine group actions through token-dependent parameters, but a four-symbol schedule may be a shallow lookup. This matters because routing cannot establish that a GDN stack must pay extra network depth or size. Closing it requires first resolving constrained realizability (G3), then an explicit resource lower bound for the parameter-generating network.

G2 — state-dependent gates are unmodeled. `CompositionalDepthDuality.gdn_state_channel_depth_bounded_in_time` and the bridge condition on a state-independent schedule, not gates that read the carried state. This matters because dynamic routing can introduce nonlinearity into that path and invalidate the affine-fold argument. Closing it requires a formal state-dependent update class and new composition and cost statements for that class.

G3 — constrained affine/reflection realizability is open. Whether the split-gated decay-minus-rank-one transition can realize a non-solvable representation, including the S_5 action, is not settled; affinity alone is no obstruction. This matters because an affine realizer could track the group with shallow token functions, while an obstruction would be algebraic rather than a temporal-depth theorem. Closing it requires either a concrete representation satisfying the production channel constraints (including allowed decay and gate ranges), or a non-realizability proof for precisely those constraints. The Lean decay parameter is an arbitrary matrix, so the formal family must not be silently identified with scalar-decay production restrictions.

O1 — the nonlinear recurrent tracker is not realized by the lookup artifact. `NDMRealizesS5.emender_realizes_s5_tracker` proves a preloaded pre-tanh generator lookup plus external multiplication of a supplied group state, not all-words execution of the recurrent tanh cell. This matters because the claimed architecture-to-task existence bridge otherwise exceeds the checked theorem. Closing Open item O1 requires an actual update configuration, initialization, decoder, and trajectory proof as specified in Section 7, set B.

These gaps are recorded in `formal/lean/GAPS-D2.md` (G1–G3) and Section 7 (O1). Circuit containment does not establish $\mathrm{TC}^0 \neq \mathrm{NC}^1$ no conclusion here assumes that open complexity separation.

Mathematical structure versus learned tracking

The composition theorem counts the nonlinear maps executed along time; it does not lower-bound the shallowest representation of their result. The conditioned affine fold does not make the full GDN2-MLP network globally linear. Neither statement identifies what optimization recovers or guarantees finite-precision robustness. Likewise, the latching hold theorem assumes $\lambda \geq 1$, outside the production decay range below one. Production-range persistence and a layer-level lift remain open.

Single-seed 1.3 B evidence and protocol boundaries

Both language-modeling arms and the current 1.3 B expressivity suite have one seed. This matters because seed variance and fine-tuning choices can change a terminal gap or its apparent length wall. Closing it requires pre-registered multi-seed repeats on the released pair with confidence intervals, common realized-budget controls, and fixed evaluation grids. The current modular-quadratic comparison shares a cap and plateau rule but realizes 2,500 versus 2,200 steps; intermediate GDN2 accuracy exceeds its terminal value, and its long-length plateau is not chance. GDN2-MLP leads the reported long-horizon S_5 comparison, and its attempted length-512 curriculum is OOM-infeasible after one optimizer step. These facts matter because protocol and hardware limits are alternative explanations; feasible matched curricula and fixed-step controls would test their contribution to the observed ordering. Large state-count headroom supports, but does not prove, a learning-failure rather than constrained-capacity explanation.

Evaluation and winner-series evidence

The 150 B pair has no held-out tokenizer-invariant bpb or downstream panel; estimated 0.93 bpb and the matched-loss null use training logs, not a held-out evaluation or an equivalence test. This matters because generalization and useful capability need not follow training-loss parity. Closing it requires running the released evaluation pipeline on both immutable checkpoints and reporting sample uncertainty.

U3.1 — carried-in winner first-generation fitness unavailable. The retained CMA snapshots do not supply the final E97-MLP/GDN2-MLP winner searches' generation-zero fitness or spread; E97′s normalized candidate/generation totals are also absent (Section 5). This matters because their within-search improvement cannot be computed or attributed to tuning from the retained rows. Closing it requires the winner searches' original generation logs and identity mapping, not imputation from earlier sweeps. Equal per-candidate rules are not proof of equal total search effort; broader, matched-total-budget searches would test the residual optimization confound.

Untested application predictions

All six applications in Section 8 are structural identifications or analogies, and every E97/GDN surrogate separation is OPEN. This matters because contraction, fixed support, alternative encodings, finite precision, and the chosen error metric can erase an apparent nonlinear-depth advantage. Closing it requires task-specific matched surrogates, long-horizon error and latency measurements, and stability/safety checks for feedback control; energy or IoT deployment claims additionally require hardware measurements. No such experiment is included here.

Scale, architecture selection, and comparators

The 1.3 B pair is a viability demonstration, not a scaling law; the single-arm 513 B-token Frontier continuation has no matched control. This matters because loss parity, memory costs, and probe ordering may change at 3–7 B. Closing it requires a matched pair across scales under one recipe. Geometry and architecture selection can also affect the observed probe gaps. Matched probe-specific searches, internal E97 ablations, and additional current-scale comparators would test those contributions. A transformer has no matched 150 B-token arm in this study; such a run would be a new comparison.

Backend and citation verification

Performance evidence is CUDA-only; direct E97 occupancy profiling and ROCm parity are not established. This matters because source portability and shared kernels do not guarantee backend correctness or throughput. Closing it requires paired ROCm correctness and performance receipts at the reported geometry, with direct E97 occupancy measurements. The bibliography audit separately flags entries whose publication metadata could not be independently retrieved; correctness-first submission requires resolving those flags, not treating citation resolution in Typst as literature verification.

11 Conclusion

Emender makes the recurrent memory itself a site of repeated nonlinear computation. A delta-correcting write uses what is already stored, and tanh transforms the state that the next token will receive. Running many small memories in parallel makes this sequential computation practical at 1.3 B parameters: E97 trains through 150.8 B Pile tokens on one eight-GPU workstation, reaching an estimated training-log 0.93 bpb. Its closely parameter-matched GDN2-MLP control occupies the same training-loss band at matched tokens; the current pair's held-out evaluation remains unavailable.

The iterated nonlinear-map experiment connects this practicality result to the architectural motivation. On the same trained pair, E97 has higher terminal accuracy and better length robustness after fine-tuning, supporting the hypothesis that repeated nonlinear state updates are a useful computational bias. The experiment uses a single seed, common cap and stopping rule, and unequal realized steps; neither model is length-invariant. The distinct S_5 group-state experiment favors GDN2-MLP at every reported long-horizon length. It measures a capability that the control learns strongly, rather than an all-purpose ranking of the models' expressivity. Additional seeds and common realized-budget controls can test how stable each finding is.

The formal results provide precise statements about the update maps and their composition. They distinguish the nonlinear state path from an affine fold under a fixed input-only schedule and relate the adjacent split-gated E97 and GDN-2 cores. They do not turn the observed probe gaps into full-stack lower bounds: recurrent tanh tracker realization remains open (O1), as does constrained affine-channel S_5 realizability (G3). These boundaries make the theoretical and empirical questions independently testable.

Feedback, iterative optimization, and dynamical systems supply concrete next tests for computation carried through memory. The examples in Section 8 identify that broader scope; they are not demonstrated deployments. The present result is a practical nonlinear recurrent architecture with comparable language-modeling loss to its matched control and an iterated-map advantage that supports the hypothesis of ongoing computation through nonlinear memory.

Reproducibility and manuscript packaging

This identified Arkheion preprint build does not establish TMLR template or anonymization compliance; journal-specific submission packaging remains a separate step.

Reproducibility. The checkpoints, loading code, and measurement scripts behind the 1.3 B pairing are public: the two immutable per-arm authorities are published as

E97 (https://huggingface.co/spinozans/emender-e97-1.3b, tag tokens-150793748480, the 150.79 B-token checkpoint of emender_E97_1.3B_20260709_084606, source .pt checkpoint SHA-256 0239706e1f67e4823008a3a2754894b5b94dc1663580d2e40c1c74f7dd6a72b2) and GDN2-MLP (https://huggingface.co/spinozans/gdn2-mlp-1.3b, tag tokens-152280498176, source .pt checkpoint SHA-256 ffae61cfeeeff820f469f7d66dd31c60e48c455872ff9102e57b15ad75bad59b), The hashes above identify the source .pt checkpoints in the release provenance, not the converted model.safetensors files. The immutable HF revisions are 119c6623f414aa7fc6e101667d3ad89b7ed6af73 (E97) and e6087fa6984265404d99c419c095692d840da472 (GDN2). Converted-weight digests are in each revision's SHA256SUMS. The releases ship with the per-architecture CMA-ES configurations, the training protocol (launcher manifests in scripts/launch_emender_8gpu_diloco.sh and scripts/launch_gdn2_mlp_8gpu_diloco.sh), and the Triton multi-programming kernel source in the public emender repo; the canonical paper PDF is mirrored at http://hypervolu.me/~erik/ndm/Garrison_2026_Emender.pdf; this reader-first alternate is a separate review-branch artifact.

12 Next tests for the current pair

The current results motivate three direct tests. First, repeat the iterated-map and S_5 protocols across seeds and common realized training budgets, recording both terminal and best-validation checkpoints. Persistence of the map advantage would strengthen the mechanism hypothesis; its disappearance under those controls would locate an optimization or protocol dependence. Second, ablate the state nonlinearity within the current E97 geometry, holding the remaining implementation and training choices fixed, to test its causal role at this scale. Third, evaluate the two immutable language-model checkpoints on held-out text and downstream tasks rather than extrapolating generalization from training loss.

13 Future Work

A matched 3–7 B E97/GDN2-MLP study would test whether the present training viability, memory cost, and computational findings persist across scale. Feedback, iterative optimization, and driven-dynamics surrogates in Section 8 provide tests beyond language modeling; their utility remains an experimental question.

The mathematical work proceeds in parallel: realize the recurrent tanh tracker itself (O1), settle constrained affine/reflection realizability (G3), and develop production-range persistence and finite-precision robustness statements. These are precise open targets, not consequences supplied by the measured probe accuracies.

APPENDIX A Development history and source register

The project developed through successive recurrent-memory designs, implementation changes, and increasingly controlled comparisons. This register records that process and points to its source material. The empirical study in this paper is the current E97/GDN2-MLP pair; earlier runs are not pooled with it or used to support its claims.

The source record remains available for provenance:

- *Architecture development.* `paper/main.typ` at base commit `f17b3be20b395cef914cd7034e8691c997164230` records the E63-to-E97 development sequence, ablations, and implementation decisions. It is an archival source for how the program evolved, not the authority for this alternate's empirical claims.
- *Exploratory task and search work.* `experiments/expressivity_tasks/`, `experiments/grok_symmetric_width/`, `paper/ndmpapernotes.md`, and `paper/results/cmaes_burst_v2/` retain development records. These separate protocols and model versions are not pooled with the current pair or treated as replications of it.
- *The present comparison.* The two immutable model releases and launcher manifests in the reproducibility section identify the actual E97 and GDN2-MLP arms. `paper/results/expressivity_1p3b/` contains their computational-probe records. The current-pair tables in this paper read the corresponding result JSONs directly.
- *Formal development.* `formal/lean/` retains the full proof project. The theorem names cited in Section 7 identify the specific statements used here; other project theorems and earlier experimental interpretations are not silently substituted for those statements.

The original records remain available in the repository. The present comparison is defined by its own model identities, data, and protocols.

References

[1] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv preprint arXiv:2312.00752*, 2024.

[2] T. Dao and A. Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," *arXiv preprint arXiv:2405.21060*, 2024.

[3] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated Delta Networks: Improving Mamba2 with Delta Rule," *arXiv preprint arXiv:2412.06464*, 2024.

[4] A. Hatamizadeh, Y. Choi, and J. Kautz, "Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention," *arXiv preprint arXiv:2605.22791*, 2026.

[5] M. Mishra, S. Tan, I. Stoica, J. Gonzalez, and T. Dao, "M²RNN: Non-Linear RNNs with Matrix-Valued States for Scalable Language Modeling." 2026.

[6] F. Danieli, P. Rodriguez, M. Sarabia, X. Suau, and L. Zappella, "ParaRNN: Unlocking Parallel Training of Nonlinear RNNs for Large Language Models," *arXiv preprint arXiv:2510.21450*, 2025.

[7] L. Gao *et al.*, "The Pile: An 800GB Dataset of Diverse Text for Language Modeling," *arXiv preprint arXiv:2101.00027*, 2020.

[8] W.-W. Liao *et al.*, "A draft human pangenome reference," *Nature*, vol. 617, no. 7960, pp. 312–324, 2023, doi: 10.1038/s41586-023-05896-x.

[9] A. Guarracino *et al.*, "Recombination between heterologous human acrocentric chromosomes," *Nature*, vol. 617, no. 7960, pp. 335–343, 2023, doi: 10.1038/s41586-023-05976-y.

[10] E. Garrison *et al.*, "Building pangenome graphs," *Nature Methods*, vol. 21, pp. 2008–2012, 2024, doi: 10.1038/s41592-024-02430-3.

[11] J. L. Elman, "Finding Structure in Time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990, doi: 10.1207/s15516709cog1402_1.

[12] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[13] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim, "Gated Linear Attention Transformers with Hardware-Efficient Training," *arXiv preprint arXiv:2312.06635*, 2023.

[14] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, "Parallelizing Linear Transformers with the Delta Rule over Sequence Length," *arXiv preprint arXiv:2406.06484*, 2024.

[15] A. S. Lahoti *et al.*, "Mamba-3: Improved Sequence Modeling using State Space Principles," in *International Conference on Learning Representations (ICLR)*, 2026.

[16] L. Feng, F. Tung, M. O. Ahmed, Y. Bengio, and H. Hajimirsadeghi, "Were RNNs All We Needed?," *arXiv preprint arXiv:2410.01201*, 2024.

[17] S. De *et al.*, "Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models," *arXiv preprint arXiv:2402.19427*, 2024.

[18] W. Merrill, J. Petty, and A. Sabharwal, "The Illusion of State in State-Space Models," *arXiv preprint arXiv:2404.08819*, 2024.

[19] W. Merrill and A. Sabharwal, "A Logic for Expressing Log-Precision Transformers," in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/a48e5877c7bf86a513950ab23b360498-Abstract-Conference.html

[20] D. Chiang, "Transformers in Uniform TC0," *Transactions on Machine Learning Research*, 2025, [Online]. Available: https://openreview.net/forum?id=ZA7D4nQuQF

[21] M. Beck *et al.*, "xLSTM: Extended Long Short-Term Memory," *arXiv preprint arXiv:2405.04517*, 2024.

[22] A. Behrouz, P. Zhong, and V. Mirrokni, "Titans: Learning to Memorize at Test Time," *arXiv preprint arXiv:2501.00663*, 2025.

[23] H. T. Siegelmann and E. D. Sontag, "On the Computational Power of Neural Nets," *Journal of Computer and System Sciences*, vol. 50, no. 1, pp. 132–150, 1995, doi: 10.1006/jcss.1995.1013.

[24] D. A. M. Barrington, "Bounded-Width Polynomial-Size Branching Programs Recognize Exactly Those Languages in NC^1", in *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing (STOC 1986)*, ACM, 1986, pp. 1–5. doi: 10.1145/12130.12131.

[25] P. Tillet, H. T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019, doi: 10.1145/3315508.3329973.

[26] A. Defazio, X. A. Yang, H. Mehta, K. Mishchenko, A. Khaled, and A. Cutkosky, "The Road Less Scheduled," *arXiv preprint arXiv:2405.15682*, 2024.

[27] A. Douillard *et al.*, "DiLoCo: Distributed Low-Communication Training of Language Models," *arXiv preprint arXiv:2311.08105*, 2023.

[28] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Automated Deduction – CADE 28*, 2021. doi: 10.1007/978-3-030-79876-5_37.

[29] The Mathlib Community, "Mathlib 4: Library Documentation." [Online]. Available: https://leanprover-community.github.io/mathlib4_docs/

[30] J. C. Candy and G. C. Temes, Eds., *Oversampling Delta-Sigma Data Converters: Theory, Design, and Simulation*. IEEE Press, 1992.

[31] S. Haykin, *Adaptive Filter Theory*, 4th ed. Prentice Hall, 2002.

[32] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Prentice Hall, 2011.

[33] I. Daubechies, M. Defrise, and C. De Mol, "An Iterative Thresholding Algorithm for Linear Inverse Problems with a Sparsity Constraint," *Communications on Pure and Applied Mathematics*, vol. 57, no. 11, pp. 1413–1457, 2004, doi: 10.1002/cpa.20042.

[34] A. Beck and M. Teboulle, "A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems," *SIAM Journal on Imaging Sciences*, vol. 2, no. 1, pp. 183–202, 2009, doi: 10.1137/080716542.

[35] H. Jaeger, "The "Echo State" Approach to Analysing and Training Recurrent Neural Networks," technical report 148, 2001. [Online]. Available: https://www.ai.rug.nl/minds/uploads/EchoStatesTechRep.pdf

[36] R. M. May, "Simple Mathematical Models with Very Complicated Dynamics," *Nature*, vol. 261, pp. 459–467, 1976, doi: 10.1038/261459a0.

[37] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Prentice Hall, 2002.

[38] B. Widrow and M. E. Hoff, "Adaptive Switching Circuits," in *IRE WESCON Convention Record, Part 4*, 1960, pp. 96–104. [Online]. Available: https://isl.stanford.edu/~widrow/papers/c1960adaptiveswitching.pdf

[39] J. Schmidhuber, "Learning to Control Fast-Weight Memories: An Alternative to Dynamic Recurrent Networks," *Neural Computation*, vol. 4, no. 1, pp. 131–139, 1992, doi: 10.1162/neco.1992.4.1.131.

[40] I. Schlag, K. Irie, and J. Schmidhuber, "Linear Transformers Are Secretly Fast Weight Programmers," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, in Proceedings of Machine Learning Research, vol. 139. 2021, pp. 9355–9366.

[41] Y. Sun *et al.*, "Retentive Network: A Successor to Transformer for Large Language Models," *arXiv preprint arXiv:2307.08621*, 2023.

[42] B. Peng and others, "RWKV: Reinventing RNNs for the Transformer Era," *arXiv preprint arXiv:2305.13048*, 2023.

[43] B. Peng and others, "RWKV-7 "Goose" with Expressive Dynamic State Evolution," *arXiv preprint arXiv:2503.14456*, 2025.

[44] Z. Qin *et al.*, "HGRN2: Gated Linear RNNs with State Expansion," *arXiv preprint arXiv:2404.07904*, 2024.

[45] M. Beck and others, "xLSTM-7B: A Recurrent LLM for Fast and Efficient Inference," *arXiv preprint arXiv:2503.13427*, 2025.

[46] J. T. H. Smith, A. Warrington, and S. W. Linderman, "Simplified State Space Layers for Sequence Modeling," *arXiv preprint arXiv:2208.04933*, 2022.

[47] F. Chaubard and M. J. Kochenderfer, "Scaling Recurrent Neural Networks to a Billion Parameters with Zero-Order Optimization," *arXiv preprint arXiv:2505.17852*, 2025.